Ajaia Docs — Product & Developer Guide

Ajaia Docs — Collaborative Document Editor MVP

Ajaia Docs is a lightweight, polished, production-style collaborative document editor inspired by Google Docs. Built as a full-stack web application, it allows team members to log in, create rich text documents, upload .txt and .md files to produce editable documents, save changes persistently in MongoDB, share documents with designated team permissions, and strictly enforce backend authorization rules (returning 403 Forbidden on unauthorized access).

Ø=Þ€ Product Features

- **Authentication & Demo Accounts**: Instant sign-in with pre-seeded demo accounts (Alex, Sarah, John) using JWT session tokens and secure password hashing (`bcryptjs`).
- **Dashboard**: Modern SaaS UI with clear separation between *My Documents* (owned) and *Shared With Me* (shared by team members), document search filtering, and clear empty states.
- **TipTap Rich Text Editor**: Clean, responsive writing interface with full support for Bold, Italic, Underline, Heading 1, Heading 2, Paragraph, Bullet lists, Numbered lists, Undo, and Redo.
- **Title Editing & Persistence**: Document titles can be edited in place and automatically persist to MongoDB.
- **Real-Time Save Indicator & Autosave**: Visual status indicator (`Saved ' `, `Saving...`, `Save failed`) driven by debounced autosave plus manual explicit Save buttons.
- **Sharing & Access Control**: Document owners can grant `EDITOR` access to other team members via an in-app Share modal.
- **Backend Authorization**: Security enforced on backend API endpoints. Unauthorized read/write requests return `403 Forbidden`.
- **File Import (.txt / .md)**: Import plain text or Markdown files (up to 5MB) to automatically instantiate editable rich text documents.
- **Automated Testing Suite**: Integration test suite verifying API authentication, document creation, document sharing, 403 authorization enforcement, and upload validation.

Ø=Þàþ Tech Stack

- **Frontend**: React 18, Vite, Tailwind CSS, TipTap Editor (`@tiptap/react`, `@tiptap/starter-kit`, `@tiptap/extension-underline`), Lucide React icons, React Router v6, Axios.
- **Backend**: Node.js, Express.js, MongoDB Atlas / Mongoose ORM (with automatic `mongodb-memory-server` fallback for zero-config local testing), Multer (file upload), JSON Web Tokens (`jsonwebtoken`), `bcryptjs`, CORS, Dotenv.
- **Testing**: Vitest + Supertest for backend integration tests.

Ø=ÜÁ Project Structure

```
llama-i/
% % %  backend/
%   % % %  src/
%   %   % % %  config/         # MongoDB & MongoMemoryServer configuration
%   %   % % %  controllers/    # Auth, Document, Share, and Upload controllers
%   %   % % %  middleware/     # JWT authentication & document authorization middleware
%   %   % % %  models/         # User, Document, and Share Mongoose schemas
```

```
%   %   % % %  routes/          # Express API route declarations
%   %   % % %  seeders/         # Seed script for Alex, Sarah, and John demo accounts
%   %   % % %  server.js        # Express app initialization & server entry point
%   % % %  tests/               # Vitest + Supertest integration test suite
%   % % %  .env.example
%   % % %  .env
%   % % %  package.json
% % %  frontend/
%   % % %  src/
%   %   % % %  components/      # ShareModal, FileUploadModal, Toast notifications
%   %   % % %  context/         # AuthContext and ToastContext
%   %   % % %  editor/          # TipTap editor & compact custom toolbar
%   %   % % %  pages/           # LoginPage, DashboardPage, EditorPage
%   %   % % %  services/        # Axios API client setup
%   %   % % %  App.jsx          # React Router setup & ProtectedRoute guards
%   %   % % %  index.css        # Tailwind CSS & custom TipTap styling
%   %   % % %  main.jsx         # App mounting
%   % % %  .env.example
%   % % %  .env
%   % % %  index.html
%   % % %  vite.config.js
%   % % %  package.json
% % %  README.md
% % %  ARCHITECTURE.md
% % %  AI-WORKFLOW.md
% % %  SUBMISSION.md
% % %  WALKTHROUGH.txt
```

Ø=Ý Seed Demo Accounts

The backend automatically seeds three demo user accounts on startup if the database is empty:

Name	Email	Password
Alex	alex@ajaia.demo	demo123
Sarah	sarah@ajaia.demo	demo123
John	john@ajaia.demo	demo123

Note: The login page includes one-click demo login buttons to quickly test multi-user sharing workflows without typing credentials.

&™þ Environment Variables

Backend (`backend/.env`)

```env
PORT=5000
MONGODB_URI=
JWT_SECRET=ajaia_docs_super_secret_jwt_key_2026
CLIENT_URL=http://localhost:5173
NODE_ENV=development
```

(If MONGODB_URI is left empty, the server automatically starts an in-memory MongoDB instance for local development and testing).

Frontend (`frontend/.env`)

```env
VITE_API_BASE_URL=http://localhost:5000/api
```

Ø<ßÃ Local Setup & Execution Guide

Prerequisites

- Node.js (v18 or higher)
- npm (v9 or higher)

1. Backend Setup & Run

```bash
cd backend
npm install
npm run dev
```

The backend server starts on http://localhost:5000 and seeds the demo users automatically.

2. Frontend Setup & Run

```bash
cd frontend
npm install
npm run dev
```

The frontend dev server starts on http://localhost:5173.

Ø>Ýê Running Automated Tests

To execute the backend integration test suite:

```bash
cd backend
npm test
```

Test Coverage Highlights:

- **Test 1**: Owner (Alex) creates document -> shares with Sarah -> Sarah requests document -> `200 OK` & can edit.
- **Test 2**: John requests Alex's document without permission -> `403 Forbidden`.
- **Test 3**: File upload rejects unsupported formats (`.pdf`) -> `400 Bad Request`.
- **Test 4**: Upload converts `.txt`/`.md` content into editable HTML document -> `201 Created`.

Ø<ß Deployment Instructions

Database (MongoDB Atlas)

1. Create a free cluster on MongoDB Atlas (https://www.mongodb.com/cloud/atlas).
2. Get the connection string and set MONGODB_URI in environment variables.

Backend Deployment (Render / Cloud Run)

1. Build command: npm install
2. Start command: npm start
3. Environment variables: Set MONGODB_URI, JWT_SECRET, CLIENT_URL, and NODE_ENV=production.

Frontend Deployment (Vercel)

1. Import frontend/ directory to Vercel.
2. Build command: npm run build
3. Output directory: dist
4. Environment variable: Set VITE_API_BASE_URL to deployed backend URL (https://your-backend.onrender.com/api).

Ø=ÜÄ Supported Upload Formats

- `.txt` (Plain text files)
- `.md` (Markdown files)
- **Maximum File Size**: 5 MB

Unsupported formats (e.g. .pdf, .docx) are gracefully rejected with a user-friendly error message.

Ø=ÜÌ Known Limitations & Intentional Scope

- **No Real-Time Collaboration**: Document updates are saved via HTTP requests and debounced autosave. Real-time WebSocket syncing (yjs/socket.io) was intentionally excluded to prioritize core CRUD, persistence, sharing, and access control within the timebox.
- **Lightweight Auth**: Uses seeded demo accounts and JWT tokens without email verification or OAuth.